1 Time to Event and Censoring

Removing censored data will result in estimation uncertainty to be larger than it could be if we were to include the censored data. Under a frequentist approach, we would lose data points which will decrease the precision of our estimates. Removing censored data could also result in biased estimates if data have only been collected for a short time. These biased estimates would be obtained from those observations where the event of interest actually happened.

Random Right-censoring: All we know is that the patient did not die due to cancer until that moment.

Left-censoring: Left-censoring happens when the event of interest occurred before you started observing someone, so you know the event time is earlier than a certain point, but you do not know exactly when.

Survival Function $S_Y(t)$ is the probability that an event has not yet occurred by time t. Standard CDF ($F_Y(t)$) measures the probability that an event has happened, the survival function measures the probability of "survival" beyond that time:$S_Y(t) = P(Y > t) = 1 - F_Y(t)$ Any survival function will always be monotonically decreasing as time goes by.

Hazard Function describes the "instantaneous" risk of an event happening at a specific moment, provided the subject has survived up to that point.

1.1 Non-parametric—Surv func with Kaplan-Meier

Training dataset of survival times, but we have no distributional assumption of survival function. This option will implicate the following: **RESTRICTED mean**: it can be estimated as the area under an estimate of the survival function. **Quantiles**: they can be estimated by inverting an estimate of the survival function.

The tidy() output provides eight metrics as columns from our training data. In the **fit_km** output, if there is no upper bound in the CI since 50% of the sampled subjects were still alive by the end of the study. See outputs of function.

```
fit_km <- survfit(formula = Surv(col1, col2) ~ 1,data = data)
tidy(fit_km) #survival function
quantile(fit_km,probs = 0.5,conf.int = FALSE) #median surv time
fit_km_plot <- autoplot(fit_km) +ylab("Survival Probability")
     +xlab("Survival Time (days)")
glance(fit_km) |> mutate_if(is.numeric, round, 3) #rmean here is
     restricted mean in glance
```

1.2 Parametric—Surv func with Weibull

We need the corresponding parameters of the distribution. Estimate the parameters of the Weibull using survreg()

Weibull: The hazard is monotonic. It either always increases, always decreases, or stays constant. If you believe your patients' risk of death only gets higher as time goes on (due to aging or disease progression), Weibull is a strong candidate.

Lognormal: The hazard is non-monotonic (arc-shaped). It typically rises to a peak and then decreases toward zero for very long-term survivors. This is common in cases where if a patient survives the initial dangerous period (like right after a high-risk surgery), their long-term risk actually drops.

```
 pweibull(q = 2, shape = 1,scale = 1,lower.tail = FALSE)
```

```
model <- survreg(formula = Surv(col1, col2) ~ 1,dist = "weibull",
     data = data) #dist="lognormal" if needed
intercept <- as.numeric(coef(model))
log_scale <- as.numeric(log(model$scale))
weibull_shape <- 1 / model$scale
weibull_scale <- exp(intercept)
median_weibull <- qweibull(p = 0.5, shape = weibull_shape, scale =
     weibull_scale)
mean_weibull <- weibull_scale * gamma(1+(1/weibull_shape))
```

1.3 Semi-Parametric-Cox Proportional Hazards Model

Do not assume any specific distribution for our response of interest. Approach in the form of a widely popular semiparametric regression model. is done through another special maximum likelihood technique using a partial likelihood. A partial likelihood is a specific class of quasi-likelihood, which does not require assuming any specific PDF for the continuous survival times.

```
 fit_ph <- coxph(formula = Surv(col1, col2) ~ j_reg, data = data)
 tidy(fit_ph, exponentiate = TRUE, conf.int = TRUE)
 profile_data <- data.frame(ph.ecog = factor(2, levels = c(0, 1,
     2)), meal.cal = 800)
fit_specific <- survfit(fit_cox, newdata = profile_data)
autoplot(fit_specific, conf.int = TRUE, surv.colour = "red") +
 labs( title = "Label", x = "Time (Days)", y = "Survival
     Probability")
```

a small enough p-value (less than the significance level) indicates that the data provides evidence in favour of association between the hazard function and the jth regressor. **Interpretation:** Now, the interpretation for the regression coefficient in a categorical regressor is the increase (at any time) by $exp(\beta_j)$ times in hazard from the baseline category to another given category.

2 Piecewise Constant Regression

Goal is an accurate prediction in many complex frameworks (e.g., non-linear). fitting different **local OLS regressions**. The **step function** breaks the ranges of the regressor into intervals. In each interval, we adjust a constant. We obtain the average response in that interval. The regression model becomes $Y_i = \beta_0 + \beta_1 C_1(X_i) + \beta_2 C_2(X_i) + \cdots + \beta_{a-1} C_{a-1}(X_i) + \beta_a C_a(X_i) + \varepsilon_i$

```
# Define the number of step functions (q + 1).
breaks_q <- 5
fat_content <- fat_content |> mutate(steps = cut(fat_content$week,
     breaks = breaks_q, right = FALSE))
levels(fat_content$steps)
model_steps <- lm(fat ~ steps, data = fat_content)
tidy(model_steps) |> mutate_if(is.numeric, round, 3)
glance(model_steps)
```

2.1 Non continuous Piecewise LR

So, for each interval beginning c1, we have an additional intercept and an additional slope.

```
model_piecewise_linear <- lm(fat ~ week * steps, data = fat_content)
```

2.2 Continuous Piecewise Linear Regression

To fix the above matter, we can impose restrictions to make sure that the lines are connected to each other. Therefore, we need to introduce the concept of the hinge function.

```
 levels(fat_content$steps)
 knots <- c(7.8, 14.6, 21.4, 28.2)
 model_piecewise_cont_linear <- lm(
   fat ~ week +
     I((week - knots[1]) * (week >= knots[1])) +
     I((week - knots[2]) * (week >= knots[2])) +
     I((week - knots[3]) * (week >= knots[3])) +
     I((week - knots[4]) * (week >= knots[4])),
   data = fat_content
 )
 glance(model_piecewise_cont_linear)
```

3 k-nn Regression

The idea behind K-NN regression is finding the K observations in our training dataset of size n (with p features x_j per observation) closest to the new point we want to predict.

```
# Use odd numbers so no missmatch on how the ties are handled.
my_knn <- knnreg(fat ~ week, data = fat_content, k = 7)
grid <- fat_content |> data_grid(week) |> add_predictions(my_knn)
```

4 (LOWESS) Regression

Predict response y for a specific x (e.g., fat content at week 10). Define Neighborhood: Identify the closest data points to the target x. The number of points (span) is a user-defined parameter. Assign Weights: Apply weights to these neighbors based on distance. Points closer to the target x receive higher weights (w_i), while distant points receive lower weights.

5 Quantile Regression

In many practical cases, we have inferential questions that are related to quantiles and not the mean. How does the alcohol price affect the weekly consumption of light drinkers, moderate drinkers, and heavy drinkers at given cut-off values of alcohol consumption? How does the alcohol price (X) affect the weekly consumption (Y) of light drinkers ($\tau = 0.25$), moderate drinkers ($\tau = 0.5$), and heavy drinkers ($\tau = 0.9$) in given cut-off values in weekly alcohol consumption? This prediction will not be a confidence interval since we are talking about probabilities (i.e., chances). Therefore, we can use "probability" instead of "confidence." Moreover, we would obtain prediction bands.

5.1 Parametric Quantile Regression

$Y_i = \beta_0(\tau) + \beta_1(\tau) X_{i,1} + \beta_2(\tau) X_{i,2} + \cdots + \beta_k(\tau) X_{i,k} + \varepsilon_i(\tau)$ where $\beta_0(\tau)$(for j = 0,1,2,..k) means that the regression term has a quantile have a different regression function on the right-hand side of our modelling equation. Unlike OLS, where there is a single model fitting, any Quantile regression approach (either parametric or non-parametric) would require fitting as many models as quantiles we are interested in! OLS method minimizes the sum of squared residuals. Its goal is to estimate the conditional mean of the response variable. Quantile Regression method minimizes an asymmetric loss function (often called the "check function" or "tilted absolute value"). Its goal is to estimate a conditional quantile (like the 75th percentile).

— When $\tau = 0.25$, we consider overpredicting. $e_i = y_i - \hat{y}_i < 0$ WORSE than underpredicting $e_i = y_i - \hat{y}_i > 0$ Note how Stepper the panalty on fidelity graph (y-axis), when we overestimate (negative part of x-axis)

— When $\tau = 0.5$, we care equally about underpredicting and overpredicting. This corresponds to the median value.

— When $\tau = 0.75$, we consider underpredicting. $e_i = y_i - \hat{y}_i > 0$ WORSE than overpredicting $e_i = y_i - \hat{y}_i < 0$

```
fit_rq_teams <- rq(formula = runs ~ hits, data = teams, tau =
     c(0.25, 0.5, 0.75))
# pairwise bootstrapping method resamples with replacement bsmethod
param_QR_summary <- summary(fit_rq_teams, se = "boot", R = 1000,
     bsmethod = "xy")
param_QR_summary[3] #75 percentile
round(predict(fit_rq_teams, newdata = data.frame(hits = 1000)), 0)
     #prediction
```

Inference: For those teams at the upper 75% threshold in runs, is this association is significant. So hits is statistically associated to the 75-quantile of runs. **Coef-Interpretation:** For each one hit increase, the 75-quantile of runs will increase by 0.5. **Prediction:** for any given team that scores 1000 hits in a future tournament, it is estimated to have a 50% (0.75-0.25*100%) chance to have between 452 and 557 runs.

5.2 Non-Parametric Quantile Regression

This class of Quantile regression implicates no model function specification. The core idea is to allow the model to capture the local behaviour of the data (i.e., capturing its local structure). Lambda decides variability. smaller lambda estimated functions will not be that smooth anymore, there will be more variability on the prediction.

```
 rqss_model_0.25_lambda_100 <- rqss(runs ~ qss(hits, lambda =
     100), tau = 0.25, data = teams)
 summary(rqss_model_0.25_lambda_100)
```

6 Missing Data

Identifying the type of missingness is critical because it determines the appropriate statistical approach. • **Missing Completely At Random (MCAR):** Missingness is totally by chance and independent of any data. It is the ideal class because there is no pattern; removing these rows does not introduce bias, though it increases standard errors due to a smaller sample size. • **Missing At Random (MAR):** Missingness depends on observed data available in the dataset. For example, a device might fail more often at night, and "time" is a recorded variable. Imputation techniques can use the observed data to fill these gaps. • **Missing Not At Random (MNAR):** The most serious type, where missingness depends on unobservable quantities. If the variable causing the missingness is not recorded, the data is MNAR, and simple deletion will likely skew the analysis and introduce serious bias. **Simple Handling Techniques & Their Drawbacks** • Listwise Deletion: Removing any row with missing data. While unbiased for MCAR, it leads to loss of power (higher standard errors) and can cause significant bias in MAR or MNAR scenarios. • Mean Imputation: Filling NAs with the overall mean of the variable. ◦ Constraint: Only for continuous or count-type variables. ◦ Drawback: It artificially reduces the standard deviation/variance of the data and ignores relationships between variables. • Regression Imputation: Uses an OLS model from complete observations to predict missing values. ◦ Drawback: It reinforces the data's correlation too perfectly, as all imputed points fall exactly on the regression line, which can "inline" the behavior artificially. multiple imputation is robust enough to work for both MAR and MNAR data

```
 md.pattern(data): Used for Exploratory Data Analysis (EDA) to
     visualize missingness patterns.
 mice(data, m = 5, method = "pmm"): Generates m imputed datasets.
 with(mids, lm(y ~ x)): Performs analysis on all imputed datasets
     simultaneously.
 pool(models): Combines multiple models into one using Rubins
     Rules.
 summary(pooled_model): Provides the final estimates, standard
     errors, and p-values
```